

零声教育 Darren

课程地址: it.0voice.com/p/t_pc/goods_pc_detail/goods_detail/course_33pRxjeNVUXYSQHvZic3iVrsm7d

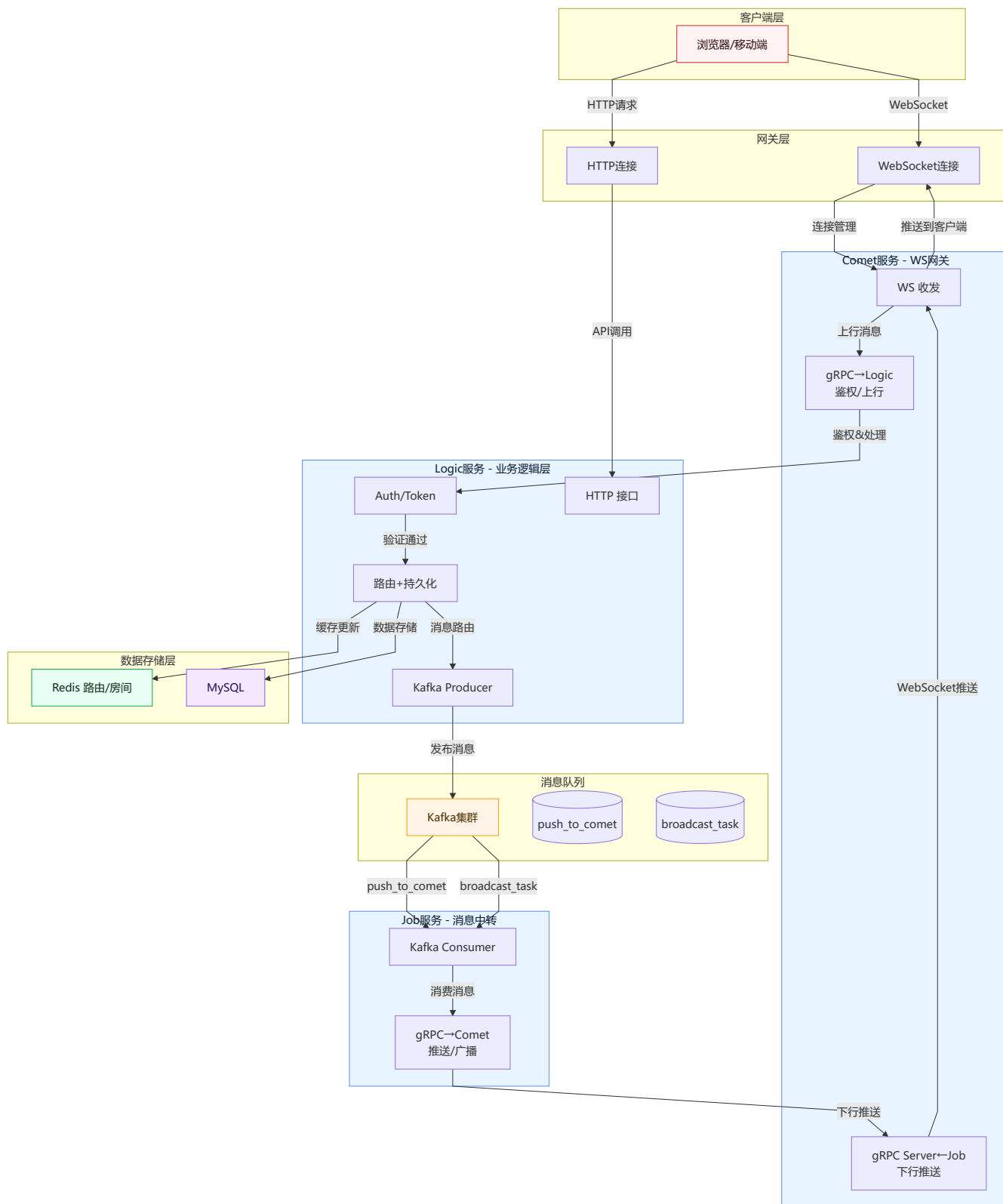
项目地址: http://gitlab.0voice.com/2510_vip/11.2-spark_push.git

下载源码, 同时让仓库从11.2-spark_push命名为 spark_push

```
git clone http://gitlab.0voice.com/2510_vip/11.2-spark_push.git spark_push
```

1 架构总览

- 角色: comet (长连接网关)、logic (业务/路由/存储)、job (Kafka 中转)、web_demo (示例客户端)。
- 通道: WebSocket (客户端 ↔ comet)、gRPC (comet → logic、job → comet)、HTTP (客户端 → logic)、Kafka (logic → job → comet)。
- 核心思路: logic 决策“消息去哪些 comet”, job 按 comet 粒度消费并推送, comet 再把消息分发到本机用户/房间的所有连接。



模块职责与交互

- **Comet:** 维护长连接/心跳, `user_id -> conn_list`, `room_id -> user_set`; 上行经 gRPC 送 logic, 下行接收 job 推送并本机 fanout。
- **Logic:** 鉴权/token; 消息持久化 (MySQL)、路由/房间缓存 (Redis); 计算 fanout → Kafka。
- **Job: Kafka consumer group**, 按 `comet_id` 将下行消息推给对应 comet (gRPC, 同步+线程池)。
- **Web Demo:** HTTP 登录、WS 收发示例页面。

线程/调用路径要点

- Comet: IO 线程只做收发, 易阻塞的 gRPC 建议放线程池或用异步 API。
- Logic: 入口 (gRPC/HTTP) → 业务线程 → MySQL/Redis → Kafka (生产异步) ; MySQL INSERT 是潜在瓶颈。
- Job: Kafka 消费线程 + 内部线程池推 gRPC; 与分区数/实例数匹配以拉高吞吐。

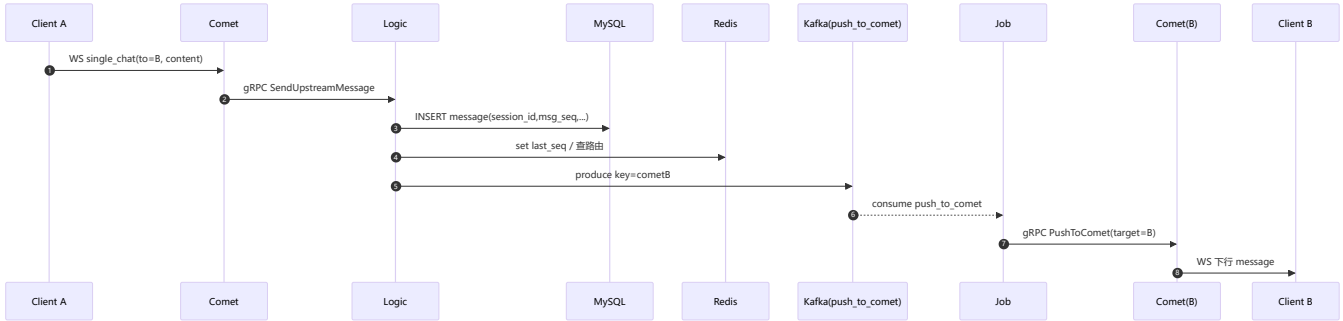
协议与接口速览

- **WebSocket (客户端 ↔Comet)**
 - 连接: ws://<host>:9000/ws?token=...
 - 上行: single_chat、group_chat/chatroom、chatroom_join/leave
 - 下行: message (单聊/群/聊天室), 可扩展 broadcast
- **gRPC**
 - Comet→Logic: VerifyToken、SendUpstreamMessage、ReportRoomJoin/Leave
 - Job→Comet: PushToComet (按用户/房间), BroadcastToAllLocalUsers (可扩展)
- **HTTP (客户端 →Logic)**
 - 登录: POST /api/login
 - 历史: GET /api/session/history
 - 聊天室: GET /api/chatroom/join|leave|online_count
 - 可选发送: POST /api/message/single|group|broadcast
- **Kafka**
 - push_to_comet: key= comet_id, value=已按 comet 拆好的消息
 - broadcast_task: 广播任务/拆分与限流

典型链路

1. 上行: WS → Comet → (gRPC) Logic → MySQL/Redis → Kafka
2. 下行: Kafka → Job → (gRPC) Comet → WS 推送给本机目标连接
3. 聊天室 (大房间) : Logic 只 fanout 到 comet, Comet 本机按房间再 fanout。

“单聊”细粒度时序



路由/房间与未读策略（项目要点）

- 用户路由: `route:user:{uid} = set<comet_id>`, 支持一用户多 comet。
- 聊天室大房间: `room:comets:{room_id}` 标记房间在哪些 comet 有在线用户; Logic 只到 comet 粒度, Comet 本机再 fanout。
- 未读: `unread = last_msg_seq - read_seq`; `last_msg_seq` 缓存在 Redis, `read_seq` 落 `user_session_state`。

典型提问 & 回答

- 为什么引入 Kafka+Job 而不是 Logic 直推? 解耦、削峰、易水平扩; 按 `comet_id` key 保序。
- 如何避免 Comet 被阻塞? 上行鉴权/发送放线程池或异步 gRPC, IO 线程只做收发。
- 消息不丢/不重复? `msg_id` 去重 (Logic/Job/Comet), Kafka 至少一次语义; `message(session_id,msg_seq)` 唯一约束。

2 协议细节（对齐 proto/spark_push.proto 与现有实现）

WebSocket 消息（客户端 ↔ Comet）

- 上行示例
 - 单聊: `{"type":"single_chat","to_user_id":123,"client_msg_id":"c1","content":{"text":"hi"}}`
 - 群/聊天室: `{"type":"chatroom","group_id":456,"client_msg_id":"c2","content":{"text":"yo"}}`
 - 房间控制: `{"type":"chatroom_join","group_id":456}` / `{"type":"chatroom_leave","group_id":456}`
- 下行示例
 - `{"type":"message","session_id":"s_1_2","msg_seq":10,"from_user_id":1,"msg_id":"m-xxx","msg_type":"text","content":{"text":"hi"}}`
- 字段对照
 - `client_msg_id`: 客户端幂等等/回执用; Logic 回传同值。
 - `session_id`: Logic 生成 (单聊 `s_<uid1>_<uid2>`, 聊天室 `r_<room_id>`)。
 - `msg_seq/msg_id`: 服务器生成; Comet/客户端用于排序/去重。

HTTP（客户端 → Logic）

- `POST /api/login` → `user_id`, `token`, `token_expire`
- `GET /api/session/history?session_id=...&anchor_seq=...&limit=...`
- `GET /api/chatroom/join|leave?room_id=...&user_id=...`
- `GET /api/chatroom/online_count?room_id=...`

- 发送 HTTP (可选) : `POST /api/message/single|group|broadcast` (与 WS 语义一致)

gRPC (见 `proto/spark_push.proto`)

- Comet→Logic
 - `VerifyToken(VerifyTokenRequest{token, comet_id}) -> VerifyTokenReply{error, user_id}`
 - `SendUpstreamMessage(UpstreamMessageRequest{from_user_id, client_msg_id, scene(single/group/chatroom/danmaku), to_user_id, group_id, video_id, timeline_ms, content_json}) -> UpstreamMessageReply{error, ChatMessage}`
 - `ReportRoomJoin/Leave(RoomReportRequest{user_id, room_id, comet_id})`
 - `UserOffline(UserOfflineRequest{user_id, comet_id})`
 - `Broadcast(BroadcastRequest{scope, group_id, content_json}) -> BroadcastReply{task_id}`
- Job→Comet
 - `PushToComet(PushToCometRequest{comet_id, ChatMessage message, repeated FanoutTarget targets})`
 - `targets` 为空: 聊天室大房间由 Comet 本机 fanout
 - `targets` 非空: 按用户列表推送

Kafka 载荷 (Logic → Job)

- Topic: `push_to_comet` (key= `comet_id`) , value = Protobuf `PushToCometRequest`
 - `comet_id`: 目标 comet
 - `message`: `ChatMessage{msg_id, session_id, msg_seq, sender_id, timestamp_ms, msg_type, content_json, client_msg_id}`
 - `targets`: 为空=房间按 comet fanout; 非空=精确用户列表
- 广播任务: Topic `broadcast_task`, value= `BroadcastTaskRequest{task_id, scope, group_id, content_json}`

Job → Comet 转发

- gRPC: `CometService.PushToComet(PushToCometRequest)`, 沿用 Kafka 载荷字段。
- Comet 处理:
 - `targets` 非空: `PushToUsers(message, targets.user_id)`
 - `targets` 为空且 `session_id` 形如 `r_<room_id>`: `PushToRoom(message, room_id)` (本机房间成员 fanout)

ChatMessage 字段 (内部统一结构)

- `msg_id`: 全局唯一 (幂等/去重)
- `session_id`: 会话 ID (单聊/群/聊天室约定)

- `msg_seq`: 会话内递增序号 (排序、未读)
- `sender_id`: 发送方
- `timestamp_ms`: 服务器生成时间戳
- `msg_type`: `text/system/danmaku/...`
- `content_json`: 消息内容 (文本 JSON)
- `client_msg_id`: 客户端自带, 端到端对应

交互样例 (均基于现有接口/字段)

- WebSocket 上行 (单聊)

Req: `{"type":"single_chat","to_user_id":2,"client_msg_id":"c-001","content":{"text":"hi"}}`

Ack: `{"type":"message","session_id":"s_1_2","msg_id":"m-abc","msg_seq":10,"from_user_id":1,"msg_type":"text","content":{"text":"hi"}}`
- WebSocket 上行 (聊天室)

Req: `{"type":"chatroom","group_id":456,"client_msg_id":"c-100","content":{"text":"room msg"}}`

Down: `{"type":"message","session_id":"r_456","msg_id":"m-room","msg_seq":88,"from_user_id":1,"msg_type":"text","content":{"text":"room msg"}}`
- WebSocket 控制

Join: `{"type":"chatroom_join","group_id":456}`; Leave: `{"type":"chatroom_leave","group_id":456}`
- HTTP 登录

Req: `POST /api/login {"account":"admin","password":"admin123"}`

Resp: `{"user_id":1,"token":"xxx","token_expire":1700000000}`
- HTTP 历史

Req: `GET /api/session/history?session_id=s_1_2&anchor_seq=0&limit=50`

Resp: `{"messages":[{"session_id":"s_1_2","msg_seq":9,"from_user_id":2,"msg_type":"text","content":{"text":"yo"}}]}`
- gRPC (SendUpstreamMessage, json 展示)

Req: `{"from_user_id":1,"client_msg_id":"c-001","scene":"single","to_user_id":2,"content_json":{"text":"hi"}}`

Rep: `{"error":{"code":0},"message":{"session_id":"s_1_2","msg_id":"m-abc","msg_seq":10,"sender_id":1,"msg_type":"text","content_json":{"text":"hi"},"client_msg_id":"c-001"}}`
- Kafka 载荷 (push_to_comet, Protobuf 展示为 json)

Key: `comet-1`

Value: `{"comet_id":"comet-1","message":{"msg_id":"m-abc","session_id":"s_1_2","msg_seq":10,"sender_id":1,"timestamp_ms":1700000000000,"msg_type":"text","content_json":{"text":"hi"},"client_msg_id":"c-001"},"targets":[{"user_id":2}]}`
- Job → Comet gRPC (PushToComet)

直接使用上面的 Value 作为请求体; targets 为空时表示聊天室由 Comet 本机按房间 fanout。

- gRPC VerifyToken
Req: {"token":"t-xxx","comet_id":"comet-1"}
Rep: {"error":{"code":0},"user_id":1}
- gRPC 房间上报 (加入/离开)
Req: {"user_id":1,"room_id":456,"comet_id":"comet-1"}
Rep: {"error":{"code":0}}
- gRPC UserOffline
Req: {"user_id":1,"comet_id":"comet-1"}
Rep: {"error":{"code":0}}
- gRPC Broadcast
Req: {"scope":"all","group_id":0,"content_json":{"text\\":\\"system msg\\"}}
Rep: {"error":{"code":0},"task_id":"task-123"}
- Redis 路由/房间
 - 用户上线: SADD route:user:1 comet-1; 下线: SREM route:user:1 comet-1
 - 房间有在线用户的 comet 集合: SADD room:comets:456 comet-1 / SMEMBERS room:comets:456
- Redis 会话/未读/缓存
 - 最新序号: SET session:last_seq:s_1_2 10
 - 热点消息缓存: LPUSH session:recent:s_1_2 <msg_json>
 - 用户会话已读 (可选缓存): HSET user_session:state:1:s_1_2 read_seq 10
 - 在线数 (Demo 近似值): SET room:online_count:456 123
- Kafka 交互补充
 - 生产: Logic 将 PushToCometRequest 写入 push_to_comet (key= comet_id, value=protobuf)
 - 消费: Job 以 consumer group (如 push_job) 拉取, 解析为 PushToCometRequest 并调用 Comet gRPC
 - 广播任务: broadcast_task 载荷 BroadcastTaskRequest; Job 可拆分为多条 PushToCometRequest 再写回 push_to_comet

3 数据库与缓存设计

MySQL 表 (核心)

表	作用	关键字段	索引/约束
user	用户账号	account 唯一	PK id, UK account
session	会话元信息 (单聊/群/聊天室)	session_id 业务 ID, type, user1_id/user2_id/group_id, last_msg_seq	PK id, UK session_id, idx user1/user2/group_id
message	消息存储	session_id, msg_seq, sender_id, msg_type, content_json	PK id, UK (session_id,msg_seq), idx session_id, sender_id

表	作用	关键字段	索引/约束
<code>user_session_state</code>	每用户会话已读游标	<code>user_id</code> , <code>session_id</code> , <code>read_seq</code>	PK <code>id</code> , UK (<code>user_id</code> , <code>session_id</code>), idx <code>session_id</code>
<code>im_group</code>	群/聊天室定义	<code>group_type</code> : 0=normal,1=chatroom,2=danmaku	PK <code>id</code> , idx <code>owner_id</code>
<code>group_member</code>	群成员	<code>group_id</code> , <code>user_id</code> , <code>role</code>	PK <code>id</code> , UK (<code>group_id</code> , <code>user_id</code>), idx <code>user_id</code>
<code>broadcast</code>	广播任务 (可选)	<code>task_id</code> , <code>scope</code> , <code>group_id</code>	PK <code>id</code> , UK <code>task_id</code> , idx <code>group_id</code>
<code>video_danmaku</code>	弹幕时间轴	<code>video_id</code> , <code>timeline_ms</code> , <code>sender_id</code>	PK <code>id</code> , idx (<code>video_id</code> , <code>timeline_ms</code>), idx <code>sender_id</code>

示例建表：见 `sql/schema.sql`（课堂可直接引用），以上表全部已在该脚本中定义。

关键字段说明

- `session.session_id`：业务层 ID，约定单聊 `s_<uid1>_<uid2>`（排序去重），聊天室 `r_<room_id>`。
- `message.msg_seq`：会话内递增序号，保证渲染有序。
- `user_session_state.read_seq`：未读 = `session.last_msg_seq - read_seq`。
- `group_type`：区分普通群、聊天室、弹幕房间，便于切换 fanout/未读策略。
- `broadcast.task_id`：业务侧广播任务 ID，幂等写入/查询用。
- `video_danmaku.timeline_ms`：播放时间轴定位（毫秒）。

Redis 设计

Key	类型	用途	说明
<code>route:user:{user_id}</code>	set	<code>user_id -></code> <code>set<comet_id></code>	支持多 comet、多连接
<code>room:comets:{room_id}</code>	set	房间在哪些 comet 有人	聊天室大房间 fanout
<code>room:online_count:{room_id}</code>	string	房间在线数（近似）	Demo 版可用
<code>session:last_seq:{session_id}</code>	string	会话最新 <code>msg_seq</code>	计算未读/历史
<code>session:recent:{session_id}</code>	list	热点消息缓存	最近 N 条
<code>user_session:state:{uid}:{sid}</code>	hash/string	热点已读缓存	可选

这些 key 均已在现有实现/设计描述中使用，没有引入新 key。

SQL/索引建议

- 保持 `message` 上的联合唯一索引 (`session_id, msg_seq`) 以避免重复写入。
- 为历史查询常用条件建立索引: `message(session_id)`, `session(session_id, type)`, `user_session_state(user_id, session_id)`。
- 大表分区/分表 (进阶): 按 `session_id hash` 或时间分表; 读写分离后历史查询走读库。

典型读写路径对应的表/Key

- 上行写入: `message` + 更新 `session.last_msg_seq` (Redis) + `session.last_msg_seq` (可选回写 MySQL)。
- 未读计算: `session.last_seq` (Redis) - `user_session_state.read_seq`。
- 聊天室路由: `room:comets:{room_id}` 优先; 若缺失可回退 `group_member` + `route:user:{uid}`。

时序与一致性 (基于现有字段)

- 写消息: Logic 分配 `msg_seq` → 写 `message` (唯一约束保证幂等) → 更新 `session.last_msg_seq` (内存/Redis, 同步或回写)。
- 未读更新: 客户端已读上报 → 更新 `user_session_state.read_seq` (MySQL+可选 Redis 缓存)。
- 路由更新: Comet 进出房间或用户上下线 → 更新 Redis `route:user:{uid}` / `room:comets:{room_id}`。

数据一致性与幂等提示

- 消息写 DB 前生成全局 `msg_id` / 业务 `msg_seq`, 重复写利用 UK 去重。
- Job/Comet 可用 LRU 集合按 `msg_id` 去重, 抵御重放。
- 弹幕/广播可选是否记录未读: 需要精确未读时建 system session; 否则只做通知。

示例查询

- 拉会话最近消息: `SELECT * FROM message WHERE session_id=? ORDER BY msg_seq DESC LIMIT 50;`
- 计算未读: 先查 Redis `session:last_seq:{sid}`, 再查 `user_session_state.read_seq`; 若缓存缺失可回落 `session.last_msg_seq`。
- 群成员列表: `SELECT user_id FROM group_member WHERE group_id=?;`

容量与索引提示

- `message` 表增长最快, 可按 `session_id` 做 hash 分表或按时间切分; 保持联合唯一索引避免重复。
- `session_id` 长度 128 已固定, 业务生成时确保格式一致 (`s_`、`r_`)。
- Redis 热 key: `route:user:*` 与 `room:comets:*`, 压测时关注 QPS 与内存。

4 依赖与编译

目前基于 Ubuntu 20.04/22.04

4.1 必备工具

- g++/clang++ (C++17) 、 cmake ≥ 3.14、 pkg-config、 git、 curl

4.2 系统库安装

```
sudo apt update

sudo apt install build-essential cmake pkg-config git curl

sudo apt install libssl-dev zlib1g-dev          # OpenSSL/压缩
# 安排protobuf和grpc相关，如果之前在grpc课程安装过环境则不需要再安装
sudo apt install protobuf-compiler libprotobuf-dev
sudo apt install libgrpc++-dev
sudo apt install protobuf-compiler-grpc

# 安装kafka客户端 sdk
sudo apt install librdkafka-dev                # Kafka client
# 安装redis客户端 sdk
sudo apt install libhiredis-dev                # Redis client
# 安装mysql客户端 sdk
sudo apt install libmysqlclient-dev           # MySQL client
# 安装json sdk
sudo apt install nlohmann-json3-dev           # JSON
```

查看某些库是否安装：

```
# 比如查看grpc
pkg-config --cflags --libs grpc++ grpc

显示：
-lgrpc++ -lgrpc
```

说明：仓库自带 muduo 源码，通过顶层 CMakeLists 自动编译；无需系统安装 muduo。

若所在发行版的 gRPC 包缺失，可选用源码编译（官方 quickstart），再通过 `cmake -DgRPC_DIR=<path>` 指向安装前缀。

4.3 kafka安装

步骤0：安装Java环境

如果还没有Java环境，则需要安装OpenJDK

Kafka 3.7.0 官方推荐使用 **Java 17**（也兼容 Java 11 或 21，但 Java 8 已不支持）。

更新系统包索引

```
sudo apt update
```

安装 OpenJDK 17 (免费开源版)

```
sudo apt install openjdk-17-jdk -y
```

如果你只需要运行 Kafka (不需要编译 Java 程序)，也可以只装 `openjdk-17-jre`，但建议直接装 `jdk`，更完整。

验证 Java 是否安装成功

```
java -version
```

正常输出类似：

```
openjdk version "17.0.12" 2024-07-16
OpenJDK Runtime Environment (build 17.0.12+7-Ubuntu-122.04)
OpenJDK 64-Bit Server VM (build 17.0.12+7-Ubuntu-122.04, mixed mode, sharing)
```

设置 JAVA_HOME 环境变量

Kafka 虽然能自动找 Java，但显式设置 `JAVA_HOME` 更稳妥。

1. 查找 Java 安装路径：

```
sudo update-alternatives --config java
```

输出示例：

```
/usr/lib/jvm/java-17-openjdk-amd64/bin/java
```

2. 提取路径前缀 (去掉 `/bin/java`)：

```
/usr/lib/jvm/java-17-openjdk-amd64
```

3. 编辑环境变量文件 (对当前用户生效)：

```
vim ~/.bashrc
```

4. 在文件末尾添加：

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
```

5. 使配置生效：

```
source ~/.bashrc
```

6. 验证:

```
echo $JAVA_HOME
```

步骤 1: 下载并解压 Kafka

```
# 下载 kafka (以 3.7.0 为例)
wget https://mirrors.huaweicloud.com/apache/kafka/3.7.0/kafka_2.13-3.7.0.tgz
tar -xzf kafka_2.13-3.7.0.tgz
cd kafka_2.13-3.7.0
```

💡 文件名中 `2.13` 是 Scala 版本, 不影响使用; `3.7.0` 是 Kafka 版本。

步骤 2: 生成 Cluster ID (仅首次需要)

KRaft 模式需要一个唯一的集群 ID:

```
bin/kafka-storage.sh random-uuid
```

输出示例:

```
C8PSUCrRLKIIG5w_D1Nug
```

复制这个 ID (每次生成是不一样的), 下一步要用。

步骤 3: 配置 Kafka (KRaft 模式)

创建配置文件 (基于模板)

```
cp config/kraft/server.properties config/kraft/server-local.properties
```

编辑 `config/kraft/server-local.properties`

```
vim config/kraft/server-local.properties
```

关键配置项 (其他保持默认即可) :

```
# 设置节点角色: 同时作为 controller + broker
process.roles=broker,controller

# 节点 ID (单机可设为 1)
node.id=1
```

```
# Controller 监听地址（必须配置！）
controller.listener.names=CONTROLLER

# 所有监听器（PLAINTEXT 用于客户端，CONTROLLER 用于内部通信）
listeners=PLAINTEXT://:9092,CONTROLLER://:9093

# 广告地址（客户端连接用）
advertised.listeners=PLAINTEXT://localhost:9092

# Controller 法定人数（单机就写自己）
controller.quorum.voters=1@localhost:9093

# 数据目录（必须是空目录！）
log.dirs=/tmp/kraft-combined-logs
```

🔔 注意：

- `controller.quorum.voters` 格式： `<node.id>@<host>:<port>`
- `log.dirs` 不能是已存在的非空目录（否则格式化会失败）

步骤 4：格式化存储目录

使用前面生成的 **Cluster ID**：

```
# 替换 YOUR_CLUSTER_ID 为你生成的实际 ID
bin/kafka-storage.sh format \
  -t YOUR_CLUSTER_ID \
  -c config/kraft/server-local.properties

# 比如：
bin/kafka-storage.sh format \
  -t C8PSucQrRLKIIG5w_DlNug \
  -c config/kraft/server-local.properties
```

✅ 成功输出：

```
Formatting storage directory /tmp/kafka-logs with metadata.version 3.7-IV2
```

步骤 5: 启动 Kafka

```
# 前台运行(先确定前台运行正常后再改为后台运行)
bin/kafka-server-start.sh config/kraft/server-local.properties

# 如果需要后台运行则(适合测试环境后台部署):
nohup bin/kafka-server-start.sh config/kraft/server-local.properties > kafka.log 2>&1 &
# 然后使用命令参考日志
tail -f kafka.log
```

看到日志中出现:

```
[ControllerServer id=1] Ready to serve as the active controller
[KafkaServer id=1] started
```

说明启动成功!

步骤 6: 验证 (创建 Topic + 生产消费)

创建 Topic

```
bin/kafka-topics.sh --create \
  --topic test \
  --partitions 1 \
  --replication-factor 1 \
  --bootstrap-server localhost:9092
```

启动消费者 (新终端)

```
bin/kafka-console-consumer.sh \
  --topic test \
  --from-beginning \
  --bootstrap-server localhost:9092
```

启动生产者 (新终端)

```
bin/kafka-console-producer.sh \
  --topic test \
  --bootstrap-server localhost:9092
```

输入消息 (如 `hello kraft`) , 看消费者是否收到。

停止 Kafka

如果是前台运行：按 `Ctrl+C` 停止服务。

如果是后台运行：

```
# 查找进程 PID
sudo ps aux | grep kafka
# 或者
sudo lsof -i:9092

# 假设 PID 是 12345
sudo kill 12345
```

下次重启只需执行：

```
bin/kafka-server-start.sh config/kraft/server-local.properties
```

✗ 不需要再次 `format`（除非你清空了 `log.dirs`）

常见问题

Q1：端口被占用？

- 修改 `listeners` 中的 `9092` 和 `9093` 为其他端口（如 `19092`, `19093`）

Q2： `log.dirs` 非空导致 `format` 失败？

- 删除目录内容：

```
rm -rf /tmp/kraft-combined-logs
```

Q3：想用 IP 而不是 `localhost`？

- 将 `localhost` 替换为你的服务器 IP（注意防火墙开放 `9092/9093`）

Q4：能否只做 `broker` 不做 `controller`？

- 可以！但单机建议合设。多机部署时才分离。

4.4 源码编译

```
cd spark_push
mkdir -p build && cd build
cmake .. -DBUILD_WEB_DEMO=ON # 可按需关掉， BUILD_WEB_DEMO默认是开启的
cmake --build . -j$(nproc)
```

生成目标：

- `logic/logic_server`
- `comet/comet_server`
- `job/job_server`
- `web_demo/web_demo_server` (若启用 `BUILD_WEB_DEMO`)
- `load_test/` 下的压测工具

4.5 可选：开发调试

- 打开断言/调试: `-DCMAKE_BUILD_TYPE=Debug`
- 关闭 Web Demo: `-DBUILD_WEB_DEMO=OFF`
- 指定安装前缀(这里目前不需要关注): `-DCMAKE_INSTALL_PREFIX=/opt/spark_push` (若后续需要 `cmake --install`)

4.6 常见编译问题排查

- 找不到依赖: 确认对应 `*-dev` 包已安装, 必要时检查 `PKG_CONFIG_PATH`、`CMAKE_PREFIX_PATH` 是否包含 `gRPC/Protobuf` 安装前缀。
- OpenSSL/cares 版本冲突: 保持系统包一致, 避免自编译混用旧版本。
- Kafka 头/库缺失: 确认 `librdkafka-dev` 安装, `pkg-config --libs rdkafka++` 可输出链接参数。

5 运行与配置

外部服务准备

导入数据库

```
# MySQL: 初始化表
cd spark_push
mysql -u root -p < sql/schema.sql
```

需要启动redis-server

```
# Redis: 默认 6379, 确保可用
```

创建kafka主题

```
# Kafka: 创建 Topic
bin/kafka-topics.sh --create --bootstrap-server 127.0.0.1:9092 \
  --replication-factor 1 --partitions 3 --topic push_to_comet
bin/kafka-topics.sh --create --bootstrap-server 127.0.0.1:9092 \
  --replication-factor 1 --partitions 3 --topic broadcast_task
```

提示：Kafka 命令需在 Kafka 安装目录或 PATH 配置好后执行；若开启 `auto.create.topics.enable=true` 可省略创建，但仍建议显式创建以控制分区数。

补充命令：

```
# 查看目前的主题
bin/kafka-topics.sh --list --bootstrap-server 127.0.0.1:9092
```

```
# 后续如果出现 链路不通时，可以用命令方式测试kafka是否正常
# 启动push_to_comet主题消费者
bin/kafka-console-consumer.sh \
  --topic push_to_comet \
  --from-beginning \
  --bootstrap-server localhost:9092

# 启动push_to_comet主题生产者
bin/kafka-console-producer.sh \
  --topic push_to_comet \
  --bootstrap-server localhost:9092
```

配置文件要点（conf/）

- `logic.conf`： `listen_port` (gRPC), `http_port`, MySQL/Redis/Kafka 地址。
 - 特别是是MySQL用户和密码要配置正确

```
# 使用实际的用户名和密码
mysql_user=root
mysql_password=123456
```

- `comet.conf`： `listen_port` (WS), `comet_grpc_port` (对 job), `logic_addr` 列表。
- `job.conf`： Kafka brokers, `comet_addr` 列表。

启动顺序

```
# 要开3个不同的终端
cd build
./logic/logic_server --config ../conf/logic.conf
./comet/comet_server --config ../conf/comet.conf # 可多实例
./job/job_server --config ../conf/job.conf
```

Web Demo

```
# 新的终端
cd build
./web_demo/web_demo_server --port 9010 --doc-root ../web_demo/static
# 浏览器访问: http://<host>:9010
比如: http://192.168.80.132:9010/
# 示例账号: admin / admin123
```

主页面:



端口对照: comet WS 9000 / comet gRPC 9100; logic HTTP 9101 / logic gRPC 9100; web_demo 默认 9010 (可改)。

快速验证

1. HTTP 登录: `POST /api/login` 获取 token。
2. WebSocket 连接: `ws://<comet_host>:9000/ws?token=...`
3. 发送单聊: 上行 `{"type": "single_chat", "to_user_id": 2, "content": {"text": "hi"}}`
4. 观察下行消息、Kafka `push_to_comet` 是否有流量、job 日志是否推送成功。

常见运行问题排查

- WebSocket 401/鉴权失败：确认 token 来自 `/api/login` 的最新值，comet→logic 的 gRPC 地址可达。
- 无下行消息：检查 Kafka `push_to_comet` 是否有生产/消费，Job 日志是否推送失败；确认 `comet_addr` 配置正确。
- MySQL/Redis 连接失败：检查 conf 中的地址/端口/密码与实际服务一致，服务是否启动。
- 端口冲突：调整 conf 文件中的 WS/gRPC/HTTP 端口，或释放占用端口后重启。

C++使用指南

三种智能指针

这里各自用法背后的动机与场景：

- `Channel` 里的 `weak_ptr` 绑定：事件回调可能在 `TcpConnection` 已被上层删除后仍到达。
`Channel::tie()` 仅保存一个 `weak_ptr`，处理事件时先 `lock()` 成 `shared_ptr`，若已过期就直接返回，避免悬垂访问，同时又不把生命周期反向延长成循环引用。

```
void Channel::handleEvent(Timestamp receiveTime)
{
    std::shared_ptr<void> guard;
    if (tied_)
    {
        guard = tie_.lock();
        if (guard)
        {
            handleEventWithGuard(receiveTime);
        }
    }
    else
    {
        handleEventWithGuard(receiveTime);
    }
}
```

- `TcpConnection` 的 `shared_ptr`：连接对象要在多个地方（事件回调、定时任务、TcpServer 的连接表）安全共享，且需要在回调里“续命”。类继承 `enable_shared_from_this`，配合别名 `TcpConnectionPtr` 统一用 `shared_ptr` 传递，确保在处理期间引用计数>0，不会被提前销毁。

```
class TcpConnection : noncopyable,
                     public std::enable_shared_from_this<TcpConnection>
```

```
typedef std::shared_ptr<TcpConnection> TcpConnectionPtr;
```

- `EventLoop` 的 `unique_ptr`：`Poller`、`TimerQueue`、`wakeupChannel` 只归 `EventLoop` 独占，生命周期与 `EventLoop` 一致，无需共享；用 `unique_ptr` 表达唯一所有权、自动析构、开销最小。

```
std::unique_ptr<Poller> poller_;  
std::unique_ptr<TimerQueue> timerQueue_;  
int wakeupFd_;  
std::unique_ptr<Channel> wakeupChannel_;
```

给面试官的讲法（避免背八股）：

- 先说场景再说类型：网络库里“事件在、对象没了”的典型坑 → 用 `weak_ptr` 先探测生存；否则就用悬垂指针或形成循环引用。
- 再讲共享需求：连接对象需要被 map 保存、回调持有、防止回调中途析构 → `shared_ptr` + `enable_shared_from_this`。
- 最后讲独占资源：事件循环内部组件只归自己，且析构顺序需确定 → `unique_ptr` 最合适，语义清晰且零额外计数成本。
- 可补一句取舍：**若只有局部短生命周期且不需所有权，可用裸指针/引用**；需要“有但不续命”就用 `weak_ptr`；需要共享就 `shared_ptr`；需要独占就 `unique_ptr`。